

On Search, Decision and the Efficiency of Polynomial-Time Algorithms

(Extended Abstract)

Michael R. Fellows* and Michael A. Langston†

Abstract. Recent advances in well-partial-order theory, especially the seminal contributions of Robertson and Seymour, have troubling consequences for those who would equate tractability with polynomial-time decidability. Specifically:

- (1) many problems are now known to be decidable in low-degree polynomial time, but only by decision algorithms with overwhelmingly astronomical constants of proportionality,
- (2) the existence of such a polynomial-time decision algorithm alone does not ensure that a corresponding search problem can be solved efficiently, and
- (3) even if both a decision problem and a corresponding search problem can be shown to be polynomial-time computable, there is no guarantee that correct algorithms can be found or even recognized within any bounded amount of time.

In this paper, we present a number of techniques for dealing with these remarkable features of algorithms based on well-partially-ordered sets. Our main results include a general strategy with which such algorithms can be made constructive. With the aid of this method, we demonstrate that low-degree polynomial-time algorithms are now *known* for almost all of the catalogued applications of RS posets. We also prove that, despite the nonconstructive nature of the well-partial-order theory on which this line of research is based, no RS poset application can settle $P \stackrel{?}{=} NP$ non-constructively by any established method of argument.

* Department of Computer Science, University of Idaho, Moscow, ID 83843. This author's research has been supported in part by the National Science Foundation under grant MIP-8603879, by the Office of Naval Research under contract N00014-88-K-0456, and by the National Aeronautics and Space Administration under engineering research center grant NAGW-1406.

† Department of Computer Science, Washington State University, Pullman, WA 99164-1210. This author's research has been supported in part by the National Science Foundation under grants ECS-8403859 and MIP-8603879, and by the Office of Naval Research under contract N00014-88-K-0343. A portion of this author's work was done while visiting the Institute for Mathematics and its Applications at the University of Minnesota, Minneapolis, MN, and while visiting the Coordinated Science Laboratory at the University of Illinois, Urbana, IL.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1989 ACM 0-89791-307-8/89/0005/0501 \$1.50

1. Introduction

Although complexity theory is formulated in terms of decision problems, established techniques of algorithm design (with rare exception [Mi]) constructively address instead corresponding search or optimization versions of the problem at hand. In the vast majority of cases in which one knows that an algorithm exists to decide a problem in polynomial time, one knows precisely what the promised algorithm is. Furthermore, if the input is a “yes” instance, such an algorithm uncovers natural evidence (that is, an answer to the search version of the problem) as the basis for a positive decision.

In contrast, recent and dramatic advances in well-partial-order theory (see, for example, [RS1-RS8]) provide new and powerful nonconstructive tools for establishing polynomial-time decidability. These deep results suffer from the following interesting difficulties:

- (1) the algorithms involve mind-boggling constants of proportionality,
- (2) the complexity of associated search problems is not established, and
- (3) there is no general means for finding or even recognizing correct algorithms.

The main purpose of this paper is to present a variety of techniques for mounting a direct and general attack on these three challenging issues.

Before moving on, we observe that one could, of course, attempt to avoid entirely all three of the difficulties listed above by taking the obvious tack, hoping to devise an alternative method for a specific problem once a polynomial-time algorithm is known to exist. From a purely practical standpoint, this may well be the most useful approach for some problems. This simplistic and parochial response to these developments is not, however, the subject of this study; we have nothing more to say about it here.

A brief background on the underlying nonconstructive tools that motivate this investigation is contained in the next section. Handling constants of proportionality is addressed in Section 3. We show how the truly gigantic structural constants of

algorithms based on the Robertson-Seymour theorems can in many cases be eliminated by bounds that focus directly on relevant structural metrics on graphs. In Section 4, we address the relationship between search, decision and oracle complexities. We exhibit a general technique for devising fast search algorithms for many problems amenable to well-partial-order theory. Finding the promised algorithms is the subject of Section 5. We demonstrate how to devise constructive search and decision algorithms in the vast majority of applications. The result is that we can, in these circumstances, *know* a low-degree polynomial-time algorithm for search (and hence decision) *without* ever knowing the (finite) obstruction set on which the existence of the decision algorithm is based. We also show that a diagonalization originally noted by Levin [Le] can be extended in a way that admits application in very general circumstances that include most (but not all) of the interesting applications of RS posets. Additionally we prove that, despite the nonconstructive nature of the underlying graph theory, this line of research cannot settle $P = NP$ nonconstructively by any usual methods of argument. The final section of this paper comprises a few conclusions and open problems that stem from this general line of research.

2. Background: Nonconstructive Tools and Their Application

Graphs we consider are finite and undirected, but may have loops and multiple edges. A graph H is less than or equal to a graph G in the *minor* order, written $H \leq_m G$, if and only if a graph isomorphic to H can be obtained from G by a series of these two operations: taking a subgraph and contracting an edge. A family F of graphs is said to be *closed* under the minor ordering if the facts that G is in F and that $H \leq_m G$ together imply that H must be in F . The *obstruction set* for a family F of graphs is defined to be the set of graphs in the complement of F that are minimal in the minor ordering. If F is closed under the minor ordering, it has the following characterization: G is in F if and only if there exists no H in the obstruction set for F such that $H \leq_m G$.

Theorem 1. [RS8] (formerly known as Wagner's Conjecture [Wa]) Any set of finite graphs contains only a finite number of minor-minimal elements. That is, graphs are *well-partially-ordered* by $\leq_m$.

Theorem 2. [RS7] For every fixed graph H , the problem that takes as input a graph G and determines whether $H \leq_m G$ is solvable in polynomial time.

If $(S, \leq)$ is a partially-ordered set for which an analog of Theorem 2 holds, we say that $(S, \leq)$ possesses *polynomial-time order tests*. We term a well-partially-ordered set with polynomial-time order tests a *Robertson-Seymour poset*, or *RS poset* for short.

Remarkably, Theorems 1 and 2 guarantee only the *existence* of a polynomial-time decision algorithm for any minor-closed family of graphs. It has been shown that Theorem 1 is independent of constructive axiomatic systems and, indeed, any proof of Theorem 1 must use impredicative methods [FRS]. Moreover, there can be no systematic method of computing the finite obstruction set for an arbitrary minor-closed family F from the description of a Turing machine that accepts precisely the graphs in F (more on this in the final section).

An interesting feature of Theorem 2 is the low degree of the polynomials bounding the decision algorithms' running times. Letting n denote the number of vertices in G , the general bound is $O(n^3)$. If F excludes a planar graph, then the bound is $O(n^2)$. These polynomials possess outrageously large constants of proportionality, rendering them impractical for problems of any nontrivial size [RS6].

A graph H is less than or equal to a graph G in the *immersion* order, written $H \leq_i G$, if and only if a graph isomorphic to H can be obtained from G by a series of these two operations: taking a subgraph and *lifting* pairs of adjacent edges. The relation $\leq_i$, like $\leq_m$, defines a partial ordering on graphs with the associated notions of closure and obstruction sets.

Theorem 3. [RS5] (formerly known as Nash-Williams' Conjecture [Na]) Any set of finite graphs contains only a finite number of immersion-minimal elements.

Theorem 4. [FL4] For every fixed graph H , the problem that takes as input a graph G and determines whether $H \leq_i G$ is solvable in polynomial time.

Theorems 3 and 4, like Theorems 1 and 2, guarantee only the existence of a polynomial-time decision algorithm for any immersion-closed fam-

ily of graphs. The method used to prove Theorem 4 yields a general time bound of $O(n^{h+3})$, where h denotes the order of the largest graph in the relevant obstruction set. With excluded-minor knowledge specific to an immersion-closed family F , however, the time complexity for determining membership can in many cases be reduced to $O(n^2)$ [FL4] by bounding the tree-width (defined later) of F .

For an application of Theorems 1 and 2, and to introduce a problem that we will address in the next section, consider GATE MATRIX LAYOUT [DKL], a combinatorial problem arising in VLSI layout. Formally, we are given an $n \times m$ Boolean matrix M and an integer k , and are asked whether we can permute the columns of M so that, if in each row we enforce the *consecutive 1s property* (that is, change to $*$ every 0 lying between the row's leftmost and rightmost 1), then no column contains more than k 1s and $*$ s.

Although the general problem is NP -complete, it has been shown that for any fixed value of k , an arbitrary instance can be mapped to an equivalent instance with only two 1s per column, then modeled as a graph on n vertices such that the family of “yes” instances is closed under the minor order and excludes a planar graph.

Theorem 5. [FL1] For any fixed k , GATE MATRIX LAYOUT can be decided in $O(n^2)$ time.

For an application of Theorems 3 and 4, and to introduce another problem that we will address in the next section, consider TWO-DIMENSIONAL GRID LOAD FACTOR, a problem that is a two-dimensional analog of MIN CUT LINEAR ARRANGEMENT [GJ]. An *embedding* of an arbitrary graph G into a fixed constraint graph C is an injection $f: V(G) \rightarrow V(C)$ together with an assignment, to each edge uv of G , of a path from $f(u)$ to $f(v)$ in C [Ro]. The *minimum load factor* of G relative to C is the minimum, over all embeddings of G in C , of the maximum number of paths in the embedding that share a common edge in C . In the TWO-DIMENSIONAL GRID LOAD FACTOR problem, we are given a graph G and integers k and w , and are asked whether the minimum load factor relative to an infinite-length, width- w grid is less than or equal to k .

Although the general problem is NP -complete even when w is fixed, it has been shown that when both k and w are fixed, the family of “yes” instances is closed under the immersion or-

der and has bounded tree-width.

Theorem 6. [FL4] For any fixed k and w , TWO-DIMENSIONAL GRID LOAD FACTOR can be decided in $O(n^2)$ time.

3. Width Metrics and Constants of Proportionality

The theory of the minor and immersion orders on finite graphs developed by Robertson and Seymour proceeds in distinct structural stages. The theorems that employ this structural information introduce stunningly enormous constants of proportionality into polynomial-time decision algorithms. These huge structural constants can sometimes be eliminated by proving problem-specific structural bounds. We focus first on what can be done for the algorithms that recognize families of graphs of bounded path-width.

Definition [RS3]. Given a graph G , a sequence $X_1, \dots, X_r$ of subsets of $V(G)$ is a *path-decomposition* of G if the following conditions are satisfied:

- (a) $\cup X_i = V(G)$,
- (b) for every edge e of $E(G)$, some X_i contains both endpoints of e , and
- (c) for $1 \leq i \leq j \leq k \leq r$, $X_i \cap X_k \subseteq X_j$.

The *path-width* of G is the minimum value of $h \geq 0$ such that G has a path-decomposition $X_1, \dots, X_r$ with $|X_i| \leq h + 1$ for $i = 1, \dots, r$.

Consider now, for example, the GATE MATRIX LAYOUT problem mentioned in the last section. It is known that, for any fixed value of k , there is a surjective map f from Boolean matrices to graphs such that all matrices mapped to the same graph have the same layout cost, the “yes” family F_k of graphs in the image of the map is minor-closed, and planar obstructions exist. Thus GATE MATRIX LAYOUT is decidable for any fixed k in $O(n^2)$ time, but with the *phenomenal* hidden structural constant c_k bounding the path-width of any graph in F_k entering into the constant of proportionality of the algorithm. (This immense constant c_k is computed by a nine step procedure that involves several compositions of towers of 2's functions. See [Jo, RS6] for details.) Happily, a direct consideration of the needed structural bound allows the constant c_k to be replaced by the “infinitely smaller” value $k - 1$.

Theorem 7. Those graphs that correspond to gate

matrix layout cost k are exactly those graphs with path-width $k - 1$.

Proof. We sketch the implication in one direction only. Suppose G has path-width $k - 1$. Then there is a path-decomposition $X_1, X_2, \dots, X_r$ that covers all edges of G , possesses the requisite intersection property, and satisfies $|X_i| \leq k$. We now build a Boolean matrix M with n rows and r columns, with column i ($1 \leq i \leq r$) representing X_i . Since M inherits the consecutive 1s property from the intersection property of the path-decomposition, and since no column contains more than k 1s, its layout cost is no more than k . We expand each column by replacing a column with j 1s with $\binom{j}{2}$ columns, each with two 1s. Letting M' denote this new ma-

trix, it follows from Lemma 4.1 of [FL1] that the layout cost of M' equals that of M . It is now easy to see that $G \subseteq G' = f(M')$, and therefore G has layout cost no more than k . $\square$

This technique can be successfully employed for a number of other minor-closed and immersion-closed problem families as well, a sampling of which follows. (For the proofs of closure, we refer the interested reader to [FL1, FL2, FL3, FL4].)

Theorem 8. The expressions listed in the table that follows bound the maximum path-width of any graph in the relevant family, and thus can replace the hidden constant in the RS algorithms for these problems.

Problem with Parameter k	Upper Bound on Path-Width
MIN CUT LINEAR ARRANGEMENT	k
MODIFIED MIN CUT ¹	$3k + 1$
SEARCH NUMBER	$k + 3$
TWO-DIMENSIONAL GRID LOAD FACTOR ²	$k(w + 1)$

¹ We measure the path-width of an input's line graph, because it is the family of line graphs that correspond to "yes" instances that is (minor-) closed.

² The parameter w denotes grid-width.

Proof. Straightforward, using arguments analogous to those applied in the proof of Theorem 7 and appropriate technical lemmas from [Bod1, MaS]. $\square$

A more general approach is to prove structural bounds specific to a particular class of obstructions. These bounds then apply to *any* family with an obstruction in that class. We now turn to tree-width, a weaker metric and the one directly used in [RS6] to bound the aforementioned structural constant c_k .

Definition [RS4]. Given a graph G , a pair (T, Y) is a *tree-decomposition* of G if T is a tree and $Y = \{X_i\}$ is a family of subsets of $V(G)$ indexed by $V(T)$ such that:

- (a) $\cup X_i = V(G)$,
- (b) for every edge e of $E(G)$, some X_i contains both endpoints of e , and
- (c) for $i, j, k \in V(T)$, if j is on the (unique) path between i and k in T , then $X_i \cap X_k \subseteq X_j$.

The *tree-width* of G is the minimum value of $h \geq 0$ such that G has a tree-decomposition (T, Y) with

$$|X_i| \leq h + 1 \text{ for } i \in V(T).$$

Clearly, for any graph G , the tree-width of G is less than or equal to the path-width of G .

Theorem 9. For any fixed $k \geq 3$, any minor-closed or immersion-closed family that excludes a cycle of length k has tree-width at most $k - 2$ and can be recognized in $O(n)$ time.

Proof. We assume that an input graph G of order n is represented with a doubly-linked adjacency list, so that standard operations (such as depth-first search) can be performed in linear time.

If G excludes a cycle of length k in either the minor or the immersion order, then G does not contain as a subgraph a cycle of length greater than or equal to k , and the number of edges of G is at most $c = (k-1)(n-1)/2 + 1$ (see Theorem III.4.9 of [Bol]). In linear time we may perform the following steps.

- Step 1. Check that there are no more than c edges in G ; if there are more, then output "no" and halt.

- Step 2. Find a depth-first spanning tree T of G .
- Step 3. Determine, for each back-edge of T , the difference between the levels of its endpoints. If any difference is as great as $k - 1$, then output “no” and halt.
- Step 4. To reach this step, it must be that every back-edge is “short.” In this case, a tree-decomposition of G can be easily read off from T (details follow).
- Step 5. Given the tree-decomposition, perform the finite list of required obstruction containment tests (the linear time bound for this step is shown in [RS7]).

To explain how Step 4 is done, suppose T is a depth-first (also called lineal) spanning tree of G for which the maximum difference between the levels of any edge’s endpoints is bounded by t . We argue that G has tree-width at most t and that a tree-decomposition witnessing this fact can be obtained from T in linear time.

We take T to be the tree that structures the decomposition, so that the family of subsets of $V(G)$ is indexed by $V(T) = V(G)$. For $v \in V(G)$, we define the subset X_v to be $\{p^0(v), \dots, p^t(v)\}$, where $p^0(v) = v$ and $p^{i+1}(v)$ is the parent of $p^i(v)$ in T (and, to handle the root r of T , we set $p^i(r) = r$ for all i).

Recalling the definition of a tree-decomposition of a graph, condition (a) is satisfied, because $v \in X_v$ for all v . Condition (b) follows from our assumption that all back-edges are sufficiently short. Condition (c) follows directly from our construction. Clearly, no member of $Y = \{X_i\}$ contains more than $t + 1$ elements. The simple nature of this decomposition allows it to be created by a single traversal of T . $\square$

Corollary. For any fixed k , VERTEX COVER [GJ], LONGEST PATH [GJ], VERTEX INTEGRITY [CEF], and EDGE INTEGRITY [FS] can be solved in $O(n)$ time.

4. Search, Decision and Oracle Complexity

Decision algorithms based on finite obstruction sets do not decide by producing (or failing to produce) natural evidence and do not solve associated search problems. The situation may be modeled in terms of relations. Associated with a relation $\Pi \subseteq \Sigma^* \times \Sigma^*$ are a number of basic computational problems that we will explore in the setting of well-partially-ordered sets:

checking – the problem of determining, for input

(x, y) , whether $(x, y) \in \Pi$,

domain decision – the problem of determining, for input x , whether there exists a y such that $(x, y) \in \Pi$, and

search – the problem of computing a search function for Π , where such a function $f: \Sigma^* \rightarrow \Sigma^* \cup \{\perp\}$ satisfies

- (1) $f(x) = y$ implies that (x, y) is in Π and
- (2) $f(x) = \perp \notin \Sigma$ implies that there exists no y for which (x, y) is in Π .

Search functions can often be computed by oracle algorithms that employ an algorithm for a related decision problem as the oracle. Such an oracle algorithm that computes a search function for Π with oracle language $\text{domain}(\Pi) = \{x \mid \text{there exists a } y \text{ for which } (x, y) \text{ is in } \Pi\}$ is termed a *self-reduction*. The *overhead* of such an oracle algorithm is defined to be just the time complexity of the oracle algorithm, where each oracle invocation is charged with only a unit-time cost.

To illustrate, consider the MIN CUT LINEAR ARRANGEMENT problem. As mentioned in Section 2, for any fixed value of k , this problem is decidable in quadratic time [FL4]. A fairly straightforward self-reduction is possible [BFL] with $O(n^2)$ overhead by “fattening up” the input graph with edges, then reading off a satisfactory permutation when any exist. Unfortunately, this scheme requires $\Theta(n^2)$ oracle calls and is therefore not *fast* as defined below.

Definition [FL6]. An oracle algorithm A with oracle language L that solves a search problem is *blind* if it has direct access only to the size of its input x . On entering the query state s_{query} , a transition is made in unit time either to state s_{yes} or to state s_{no} according to whether $q.x \in L$, where q denotes the contents of the query register.

In the case of graph problems, where the size of the input is the number of vertices, we may assume that the algorithm knows that the vertex set is $\{1, \dots, n\}$, and that it may refer to these vertices unambiguously when consulting the oracle. This arrangement readily generalizes to other kinds of problems.

Definition [FL6]. The *blind oracle complexity* of a search problem Π_S is the function $\min(\max_A(n))$ where A is any blind oracle algorithm solving Π_S .

and $\max_A(n)$ is the maximum number of oracle calls made by A on any input of length n .

This definition of blind oracle complexity generalizes, in some respects, the concept of *elusiveness* of graph properties (see [Bol] for an exposition).

Definition [FL6]. An oracle algorithm A for solving the search problem for a relation Π is *fast* if the number of calls made by A is within a constant factor of the blind oracle complexity of Π .

We have previously employed adversary arguments to establish that the blind oracle complexity of fixed- k MIN CUT LINEAR ARRANGEMENT is $\Omega(n \log n)$. We have also introduced a technique that we termed *scaffolding* to design a blind oracle algorithm requiring only $O(n \log n)$ oracle calls, where the calls are to a decision algorithm with complexity $O(n^2)$.

Theorem 10 [FL6]. For any fixed k , the blind oracle complexity of MIN CUT LINEAR ARRANGEMENT is $\Theta(n \log n)$.

The lower bound arguments used in the proof of Theorem 10 do not apply to non-blind oracle reductions. The following result shows why, thereby answering an open problem posed in [FL5].

Theorem 11. For any fixed k , a satisfactory solution to MIN CUT LINEAR ARRANGEMENT can be constructed, when any exist, by an oracle algorithm with $O(n)$ overhead that makes $O(n)$ oracle calls.

Proof. We assume that the input graph is connected and exploit its structure in two ways: first, no vertex can have a degree exceeding $2k$; second, an edge with long bandwidth in a layout reduces further the maximum degree of vertices embedded between the edge's endpoints.

With $\lceil \log_2 n \rceil$ oracle calls we can locate the position of some arbitrary vertex, v , in a solution permutation. (Of course, we cannot iterate this simple scheme, since this would require $O(n \log n)$ calls.) We next select an unplaced neighbor of v , say w , and question the oracle about its direction and *displacement* from v . If (v, w) has long bandwidth in every solution permutation, then the number of questions could be quite large, even $O(\log n)$. In an *amortized* sense, however, this is

fine: any vertex x embedded between v and w has degree at most $2k - 2$, rather than $2k$. (Clearly, this generalizes so that if x is embedded between the endpoints of h distinct edges, then x has degree at most $2k - 2h$.) Thus it follows that an initial budget of $O(k)$ calls per vertex is sufficient. $\square$

Theorem 11 places no restriction on the power of the oracle, but in fact the oracle language can be recognized in $O(n^2)$ time by a reduction based on a scaffolding strategy to a decision problem amenable to RS posets. This general strategy yields efficient search algorithms for problems that satisfy the following *uniformity condition* concerning the complexity of the associated fixed-parameter decision problem: there is a constant c such that for *any* fixed value of k , membership can be decided in $O(n^c)$ time. See [FL5, FL6] for more details on scaffolding.

Theorem 12. For any fixed k , a satisfactory solution to MIN CUT LINEAR ARRANGEMENT can be constructed, when any exist, by an oracle algorithm with $O(n)$ overhead that makes $O(n)$ calls to a decision oracle whose complexity is $O(n^2)$.

Proof. Analogous to the proof of Theorem 11, now using a scaffold that is a simple path of length $n - 1$, with k copies of each edge in the path. We set $k' = 2k$ and use a decision algorithm for MIN CUT LINEAR ARRANGEMENT, with (fixed) parameter k' , as the oracle. Thus the scaffold cannot be “kinked.” A vertex is placed in the layout by *identifying* it with a vertex of the scaffold. After an arbitrary vertex v is placed at some location i , $1 \leq i \leq n$, we place a neighbor of v using the location query sequence $i \pm 1, i \pm 2$, and so on. $\square$

The techniques used in the proofs of Theorems 11 and 12 directly yield similar results for a number of additional problems amenable to well-partial-order theory, including those problems mentioned in Theorem 8. (We leave the task of proving this to the curious reader. The choice of k' and the details of the scaffold are specific to the particular problem under consideration [FL5, FL6].) For problems such as MIN CUT LINEAR ARRANGEMENT, therefore, the overall time complexity has been considerably reduced: from an upper bound of $O(n^{k-1})$ as derived in [MaS] to a bound of $O(n^3)$ as guaranteed by the last theorem. Moreover, it has very recently been reported to us that the technique

we have presented above can be extended to reduce the bound even further to $O(n^2)$ with a clever scheme that exploits the repeated use of a single tree-decomposition [Bod2]. It is an open question whether subquadratic-time algorithms are possible.

5. On Transforming Existence Into Construction

It is a remarkable development that we now have powerful tools for proving the existence of low-degree polynomial-time decision algorithms for many natural problems, and yet there can be no general method for producing the finite obstruction sets on which these algorithms are based. Despite the discouraging inherent nonconstructivity of these tools, however, we will show in this section that, in most of the presently known applications, low-degree polynomial-time search (and hence decision) algorithms *can* be constructed. The general technique we will present works in a rather surprising fashion: we will be able to write down a correct algorithm without knowing the complete relevant obstruction set and, in some cases, without knowing the exact polynomial that bounds the running time of the algorithm! In fact, we will prove something more than this, as follows.

Definition. A partial order $(R, \leq)$ is *uniformly enumerable* if there is a recursive enumeration $(r_0, r_1, r_2, \dots)$ of R with the property that $r_i \leq r_j$ implies $i \leq j$.

In the minor and immersion orders, for example, a natural uniform enumeration is to generate all finite graphs based on a monotonically non-decreasing sequence of number of vertices, with graphs having the same number of vertices generated based on a monotonically nondecreasing sequence of number of edges, with ties (graphs with the same number of vertices and the same number of edges) broken arbitrarily.

Definition. Under a uniform enumeration of the elements of $\text{domain}(\Pi)$, we say that a self-reduction algorithm for Π is *uniform* if, on input r_j , the oracle for $\text{domain}(\Pi)$ is consulted concerning r_i only for $i \leq j$.

Definition. An oracle algorithm is *honest* if, on inputs of size n , its oracle is consulted concerning only instances of size $O(n)$.

Definition. An oracle algorithm A with overhead bounded by $T(n)$ is *robust* (with respect to $T(n)$) if A is guaranteed to halt within $T(n)$ steps for any oracle language.

Theorem 13. Let $F = \text{domain}(\Pi)$ be a closed family in a uniformly enumerable well-partial-order, and suppose the following are known:

- (1) an algorithm that solves the checking problem for Π in $O(T_1(n))$ time,
- (2) order tests that require $O(T_2(n))$ time,
- (3) a uniform self-reduction algorithm (its time bound is immaterial), and
- (4) an honest robust self-reduction algorithm that requires $O(T_3(n))$ overhead.

Then an algorithm requiring $O(\max\{T_1(n), T_2(n) \cdot T_3(n)\})$ time is known that solves the search problem for Π .

Proof. Let I denote an arbitrary input instance and let K denote the known elements of the (finite) obstruction set for F . (Initially, K can be empty.) We treat K as if it were the correct obstruction set, that is, as if we *did* know a correct decision algorithm. Since the elements of K will always be obstructions, if we find that $H \leq I$ for some $H \in K$, then our algorithm reports “no” and halts. Otherwise, after checking each element of K to confirm that it is not less than or equal to I , we attempt to self-reduce and check the solution so obtained. If the solution is correct, then our algorithm reports “yes,” outputs the solution and halts.

In general, however, it may turn out that our check of the solution reveals that we have self-reduced to a non-solution. This can only mean that there is at least one obstruction $H \notin K$ such that the index of H in the uniform enumeration of R is less than or equal to the index of I (this does not necessarily mean that $H \leq I$). In this event, we proceed by generating the elements of R in uniform order until we find a new obstruction (an element that properly contains no other obstruction but that cannot be uniformly self-reduced to a solution). When such an obstruction is encountered, we need only augment K with it and start over on I .

Let C_1 denote the cardinality of the correct obstruction set and let C_2 denote the largest number of vertices in any of its elements. Then, for some suitably chosen function f , the total time spent by this algorithm is bounded by $O(C_1(T_1(n) + T_2(n) \cdot T_3(n)) + f(C_2))$. $\square$

Theorem 13 prompts the following observations.

Observation 1. Most (but, interestingly, not all) of the known RS poset applications that insure polynomial-time problem complexity can be made constructive. This follows because polynomial-time order tests are known for the minor and immersion orders and because we know, in most cases, low-degree polynomial-time algorithms for checking and for (uniform and honest robust) self-reducing. For example, let us consider once more the MIN CUT LINEAR ARRANGEMENT problem. Checking a candidate solution is easily performed in $O(n)$ time. Uniform self-reduction is achievable, although it is rather cumbersome. (We leave the explicit construction to the inquiring reader, with the hint that one need only expand slightly the brute-force component of the nonuniform self-reduction reported in [BFL].) The oracle-reduction employed in Theorem 12 is in fact an honest robust self-reduction for special instances of a closely-related search problem, exploiting the aforementioned uniformity condition. Therefore, despite the notoriously nonconstructive nature of the underlying theory as described in Section 2, Theorems 6, 12 and 13 guarantee the following constructive corollary: for any fixed k , the search and decision versions of MIN CUT LINEAR ARRANGEMENT can be solved in $O(n^3)$ time with a *known* algorithm.

Observation 2. For problems such as KNOT-LESSNESS [FL2], for which no algorithm (with any time bound) for the domain decision problem is constructively known, but for which (only super-exponential) algorithms for the checking problem are constructively known [Ha], we have the rather unexpected result that performing uniform self-reduction may be easier than decision. That is, if one knew an algorithm for uniform self-reduction and if its use in Theorem 13 provided the only known decision algorithm, then (this method of) decision clearly takes longer than self-reduction.

Observation 3. In Theorem 13, it is of course possible to replace the hypothesis that a uniform self-reduction is known with the alternate hypothesis that a domain decision algorithm is known. (In fact, due to Theorem 13 itself, it follows that this new hypothesis is potentially weaker!) In some applications, this may be more convenient.

Observation 4. Although, at first blush, any attempt to implement the algorithm used in the proof of Theorem 13 appears to be completely out of the question, closer inspection reveals that it may in fact provide the basis for viable and wholly novel approaches for solving a number of practical problems. First, note that this scheme can be viewed as a “learning algorithm” that gradually accumulates a useful subset of the obstruction set, and invokes its exhaustive “learning component” only when forced to do so by input it cannot handle with this subset. Furthermore, some obstructions are already known or are easy to identify for most problems, so that we need not start with the empty set. There is a growing body of evidence to indicate that, for many problems amenable to RS poset theory, a relatively small collection of obstructions is often enough to support search and decision algorithms on input generally encountered in practice (see [FKL] for more details).

Our next result extends a novel and theoretically entertaining (but completely unimplementable) idea first observed by Levin [Le] and explicitly proved by Schnorr [Sc] to the setting of well-partially-ordered sets. The original idea applies to the computation of search functions restricted to $\text{domain}(\Pi)$. Where $\text{domain}(\Pi)$ is closed in a well-partial-order, this restriction can be lifted.

Theorem 14. Let $F = \text{domain}(\Pi)$ be a closed family in a uniformly enumerable well-partial-order, and suppose the following are known:

- (1) an algorithm that solves the checking problem for Π in $O(T_1(n))$ time,
- (2) order tests that require $O(T_2(n))$ time, and
- (3) a uniform self-reduction algorithm (its time bound is immaterial).

Then an algorithm requiring $O(\max\{T_0(n) + T_1(n) \cdot \log T_0(n), T_2(n)\})$ time is known that solves the search problem for Π , where $T_0(n)$ denotes the time complexity of *any* algorithm solving this search problem.

Proof. Condition (3) ensures that at least one search algorithm exists, so that T_0 is defined. We interleave the following two operations. In case the input is in F , we employ Levin’s exponential form of diagonalization, which requires time proportional to $2^X T_0(n) + (X + \log T_0(n)) T_1(n)$, where X denotes the index of the lowest-indexed Turing machine solving the search problem in time $T_0(n)$.

In case the input is in $\overline{F}$, we employ the uniform enumeration of the elements of the order (along with obstruction containment tests) as we did in the proof of Theorem 13. $\square$

Thus one theoretically attains asymptotic optimality, albeit at the cost of unfathomably explosive constants of proportionality. A curious feature of the preceding theorem is that provision is made neither for computing the relevant obstruction set nor for determining the function T_0 . That is, even if one could implement the algorithm, no bounded amount of computation would necessarily reveal when one has “captured” the last obstruction or when an optimal algorithm begins to “outshine” all others.

To preface our final result of this section, we observe that many members of the research community have invested considerable effort in producing NP -completeness proofs for a vast array of seemingly difficult combinatorial problems. Such proofs inherently rely on the assumption that, if $P = NP$, an NP -completeness proof is not in vain, but instead becomes a polynomial-time algorithm. There has, nevertheless, always been a potentially-fatal flaw in this logic. Recent advances such as those described in Section 2 now make this concern obvious indeed. As voiced in [Jo], the problem is that a proof of $P = NP$ might be nonconstructive. For example, what if CHROMATIC NUMBER [GJ] with, say, fixed $k = 3$ were minor-closed (which it is not)? After all, traditional methods for attacking the $P = NP$ question have to date failed completely; one might expect that if the issue is ever to be resolved, dramatically new and powerful techniques (and tools from well-partial-order theory surely qualify on that point) must be brought to bear. We now show that such a vexing outcome cannot happen based on well-partially-ordered sets and known problem reduction schemes. Recall that an RS poset is a well-partially-ordered set that supports polynomial-time order tests. We make the additional assumption that these order tests are known (as they are in the minor and immersion orders).

Definition. By the statement P is *constructively equal to NP* we mean that an algorithm is known that computes, from the index of a nondeterministic polynomial-time Turing machine that recognizes a set X , the index of a deterministic polynomial-time Turing machine that recognizes X . A set X is *constructively NP -hard* if a

polynomial-time many-to-one reduction from SAT to X is known.

Every problem presently known to be NP -hard is constructively NP -hard as well, simply because the relevant reductions are constructively known (rather than only known to exist). We now demonstrate that it is not possible to prove $P = NP$ by an obvious approach, such as searching for a known NP -complete graph problem that can be shown to be minor-closed, except in a particularly strong (constructive) way.

Theorem 15. Let F denote a closed family in a uniformly enumerable RS poset. If it is constructively NP -hard to determine membership in F , then P is constructively equal to NP .

Proof. Let i denote the index of a nondeterministic polynomial-time Turing machine M_i with $L(M_i) = L$. We compute the index i' of a deterministic polynomial-time Turing machine that accepts L by describing a polynomial-time algorithm for recognizing L . For input x , we can of course compute, by a known algorithm, in time polynomial in $|x|$ a Boolean expression E_x that is satisfiable if and only if $x \in L$. It is enough to argue that knowing a reduction f from SAT to F -membership yields a known polynomial-time algorithm for SAT .

Note that an honest robust (and uniform) polynomial-time self-reduction algorithm for SAT is easily described, by simply taking the usual self-reduction and guarding against a faulty oracle, precomputing the number of oracle calls required when the oracle is trustworthy.

Let Im^+ denote the set of all graphs that are images (under the many-to-one reduction f) of satisfiable expressions, and let Im^- denote the set of graphs that are images of unsatisfiable expressions. Thus, $Im^+ \subseteq F$ and $Im^- \subseteq \overline{F}$. Since F is closed, Im^+ is closed in $Im = Im^+ \cup Im^-$, under the inherited order. Let O_{SAT} denote the minimal elements of Im^- . The set O_{SAT} is finite, by the well-partial-ordering of an RS poset.

Let E denote an arbitrary Boolean expression. Clearly, E is satisfiable if and only if $f(E) \not\leq y$ for each $y \in O_{SAT}$. Let $E_1, E_2, \dots$ be a recursive enumeration of all Boolean expressions, and let O denote the known candidates for O_{SAT} . (Initially, O can be empty.) Each element of O will be greater than or equal to some obstruction, so we treat O as if it were O_{SAT} . We begin by gener-

ating expressions in the enumeration and exhaustively determining whether each is satisfiable until we encounter an E_j that is unsatisfiable. Resetting O to be the minimal elements of $O \cup \{f(E_j)\}$, we use the known order tests to learn whether $f(E)$ contains an element of O . If it does, then our algorithm reports “no” and halts. Otherwise, we attempt to self-reduce E using f together with the order tests to implement the oracle. If we succeed in producing a satisfying truth assignment, then our algorithm reports “yes” and halts. If the question remains unsettled (that is, the attempted self-reduction has failed, but there is no $y \in O$ for which $f(E) \geq y$), then we resume generating expressions until we can augment O and start over on E .

Since O_{SAT} is finite, we are guaranteed to achieve $O = O_{SAT}$ within some bounded initial segment of the enumeration, although evidence for a correct decision concerning E may be produced well before that point is reached. The running time of the algorithm is bounded by a polynomial function of $|x|$. $\square$

Incidentally, while the method we used to prove the last result can be viewed as an extension of the technique we employed in the proof of Theorem 13, we note that an alternate proof of Theorem 15 has very recently been devised by Cook and Gupta [CG] using the notion of sparse sets.

6. Concluding Remarks

Our constructivization techniques do not depend on knowing, nor do they provide a means for computing, the relevant (finite) obstruction sets. An obvious question arises: can these sets be systematically computed? The following theorem shows that, in a general sense, they cannot. (We state this result for the minor ordering of finite graphs; the proof can be easily modified to handle other RS posets.)

Theorem 16. There is no algorithm to compute, from a finite description of a minor-closed family F of graphs as represented by a Turing machine that accepts precisely the graphs in F , the set of obstructions for F .

Proof. We reduce from the Halting Problem. Given a Turing machine M and a word x , we can determine whether M halts on x as follows. We modify the description of M to obtain a description of a Turing machine M' that embodies the

following algorithm. Given as input a graph G , M' first computes the index $i(G)$ of G in a recursive enumeration of all finite graphs that is uniform with respect to the minor order. M' then simulates M on input x for $i(G)$ steps.

If M does not halt in at most $i(G)$ steps, then M' accepts G . Otherwise, if M halts in exactly $i(G)$ steps, then M' rejects G . If the halting time of M on input x is $t < i(G)$, then M' determines the graph H such that $i(H) = t$, and tests whether $G \geq_m H$. If $G \geq_m H$, then M' rejects G . Otherwise, M' accepts G .

We claim that M' accepts a minor-closed family of graphs. If M does not halt on input x , then M' accepts all graphs, which is trivially minor-closed. If the halting time of M on input x is t , then M' accepts precisely those graphs G for which $G \not\geq_m H$, a minor-closed family with the single obstruction H . To see this, suppose $G \geq_m H$. Since the enumeration is uniform, $i(G) \geq t = i(H)$, and so G is rejected by M' . If $G \not\geq_m H$, then G is accepted by M' , regardless.

The description of M' is clearly computable from x and the description of M . Whether M halts on x can thus be determined by computing the obstruction set of the family of graphs accepted by M' , since this obstruction set is empty if and only if M fails to halt on input x . $\square$

Although no general method is known for identifying an arbitrary obstruction set and, by the above theorem, no completely general method is possible, special arguments and heroic case analyses (for example, [Ki, MRS]) can sometimes be applied to identify sets for individual problems.

A central open question that the results we have reported here address, but by no means completely resolve, is whether algorithms based on well-partially-ordered sets can be made effective in practice. We have demonstrated that for many of the presently known RS poset applications the problem of astronomical constants can be greatly mitigated. We have also devised efficient oracle-reduction schemes for large classes of problems that are amenable to this approach, and note that the search for fast self-reduction and oracle-reduction strategies continues to be an active area of research for other important problems (for example, GRAPH GENUS [AFL]). Perhaps the most interesting result we have developed in this paper is a general method of constructivization that yields known low-degree polynomial-time search (and hence decision) algorithms where pre-

viously only the existence of low-degree decision algorithms was proved. We are optimistic that this technique may support effective learning strategies based on the notion of approximate obstruction sets. Nevertheless, although we have made substantial progress in resolving these three stumbling blocks to any implementable algorithmic strategy based on recent breakthroughs in well-partial-order theory, it remains to be seen whether this line of research will one day make truly practical algorithms available.

Acknowledgements.

We wish to thank Vijaya Ramachandran for bringing to our attention Leonid Levin's original idea. We also wish to express our appreciation to those who have enthusiastically participated in a number of lively technical discussions concerning this subject, including Dan Archdeacon, Mike Attalah, Manuel Blum, Donna Brown, Randy Bryant, Amos Fiat, Victor Klee, Gene Lawler, Dave Liu, John Reif, Neil Robertson, Arny Rosenberg, Paul Seymour and many others.

References

- [AFL] D. Archdeacon, M. R. Fellows and M. A. Langston, "Fixed Genus Embedding in $O(n^4)$ Time," in preparation.
- [BFL] D. J. Brown, M. R. Fellows and M. A. Langston, "Polynomial-Time Self-Reducibility: Theoretical Motivations and Practical Results," *Int'l J. of Comp. Math.*, to appear.
- [Bod1] H. L. Bodlaender, "Classes of Graphs with Bounded Tree-Width," Technical Report RUU-CS-86-22, Department of Computer Science, University of Utrecht, 1986.
- [Bod2] H. L. Bodlaender, "Improved Self-Reduction Algorithms for Graphs with Bounded Tree-Width," Technical Report RUU-CS-88-29, Department of Computer Science, University of Utrecht, 1988.
- [Bol] B. Bollobás, *Extremal Graph Theory*, Academic Press, New York, 1978.
- [CEF] L. Clark, R. C. Entringer and M. R. Fellows, "Computational Complexity of Integrity," *J. Combin. Math. Combin. Comput.* 3 (1988), 549-560.
- [CG] S. Cook and A. Gupta, private communication.
- [DKL] N. Deo, M. S. Krishnamoorthy and M. A. Langston, "Exact and Approximate Solutions for the Gate Matrix Layout Problem," *IEEE Trans. on Computer-Aided Design* 6 (1987), 79-84.
- [FKL] M. R. Fellows, N. G. Kinnersley and M. A. Langston, "Finite-Basis Theorems and a Computation-Integrated Approach to Obstruction Set Isolation," *Proc. Computers and Mathematics Conf.* (1989), to appear.
- [FL1] M. R. Fellows and M. A. Langston, "Nonconstructive Advances in Polynomial Time Complexity," *Info. Proc. Letters* 26 (1987), 157-162.
- [FL2] ———, "Nonconstructive Tools for Proving Polynomial-Time Decidability," *J. of the ACM* 35 (1988), 727-739.
- [FL3] ———, "Layout Permutation Problems and Well-Partially-Ordered Sets," *Proc. 5th MIT Conf. on Advanced Research in VLSI* (1988), 315-327.
- [FL4] ———, "On Well-Partial-Order Theory and Its Application to Combinatorial Problems of VLSI Design," Computer Science Technical Report CS-88-188, Washington State University, 1988.
- [FL5] ———, "Fast Self-Reduction Algorithms for Combinatorial Problems of VLSI Design," *Proc. 3rd Aegean Workshop on Computing* (1988), 278-287.
- [FL6] ———, "Fast Search Algorithms for Graph Layout Permutation Problems," Computer Science Technical Report CS-88-189, Washington State University, 1988.
- [FRS] H. Friedman, N. Robertson and P. D.

- Seymour, "The Metamathematics of the Graph Minor Theorem," in *Applications of Logic to Combinatorics*, American Math. Soc., Providence, RI, to appear.
- [FS] M. R. Fellows and S. Stueckle, "The Immersion Order, Forbidden Subgraphs and the Complexity of Integrity," to appear.
- [GJ] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, CA, 1979.
- [Ha] W. Haken, "Theorie der Normalflächen," *Acta Mathematica* 105 (1961), 245–375.
- [Jo] D. S. Johnson, "The NP-Completeness Column: An Ongoing Guide," Column 19, *J. Algorithms* 8 (1987), 285–303.
- [Ki] N. G. Kinnersley, "Obstruction Set Isolation for Layout Permutation Problems," Ph. D. Thesis, Washington State University, to appear.
- [Le] L. A. Levin, "Universal Enumeration Problems," (Russian) *Problemy Peredaci Informacii*, Tom IX (1972), 115–116.
- [MaS] F. S. Makedon and I. H. Sudborough, "On Minimizing Width in Linear Layouts," to appear.
- [Mi] G. L. Miller, "Riemann's Hypothesis and Tests for Primality," *J. of Computer and Systems Science* 13 (1976), 300–317.
- [MRS] R. Motwani, A. Raghunathan, and H. Saran, "Constructive Results from Graph Minors: Linkless Embeddings," *Proc. 29th IEEE Symp. on Foundations of Computer Science* (1988), 398–409.
- [Na] C. Nash-Williams, "On Well-Quasi-Ordering Infinite Trees," *Proc. Cambridge Phil. Soc.* 61 (1965), 697–720.
- [Ro] A. Rosenberg, "Issues in the Study of Graph Embeddings," *Lecture Notes in Computer Science* 100 (1981), 150–176.
- [RS1] N. Robertson and P. D. Seymour, "Disjoint Paths—a Survey," *SIAM J. Alg. Disc. Meth.* 6 (1985), 300–305.
- [RS2] ———, "Graph Minors—a Survey," in *Surveys in Combinatorics* (I. Anderson, ed.), Cambridge Univ. Press, 1985.
- [RS3] ———, "Graph Minors I. Excluding a Forest," *J. Comb. Th. Ser. B* 35 (1983), 39–61.
- [RS4] ———, "Graph Minors II. Algorithmic Aspects of Tree-Width," *J. Algorithms* 7 (1986), 309–322.
- [RS5] ———, "Graph Minors IV. Tree-Width and Well-Quasi-Ordering," *J. Comb. Th. Ser. B*, to appear.
- [RS6] ———, "Graph Minors V. Excluding a Planar Graph," *J. Comb. Th. Ser. B* 41 (1986), 92–114.
- [RS7] ———, "Graph Minors XIII. The Disjoint Paths Problem," to appear.
- [RS8] ———, "Graph Minors XVI. Wagner's Conjecture," to appear.
- [Sc] C. P. Schnorr, "Optimal Algorithms for Self-Reducible Problems," *Proc. ICALP* (1976), 322–337.
- [Wa] K. Wagner, "Über Einer Eigenschaft der Ebener Complexe," *Math. Ann.* 14 (1937), 570–590.